online-kitchen Technical Documentation

1. Project Overview

online-kitchen (also referred to as Digital Kitchen) is a backend food ordering system built with a robust, scalable architecture. It provides a complete end-to-end solution for online food ordering, including:

â¢ **User Authentication & Management**: Secure registration and login using JWT, with role-based access control (User/Admin).
â¢ **Food Menu Management**: A dynamic menu system where admins can manage food items, categories, and availability.
â¢ **Cart Operations**: Real-time management of user carts, allowing items to be added, updated, or removed before checkout.
â¢ **Order Processing**: A structured ordering flow that validates item availability, calculates totals, and tracks order status from "Pending" to "Delivered".

2. Tech Stack

â¢ **Runtime**: [Node.js](https://nodejs.org/)
â¢ **Framework**: [Express.js](https://expressjs.com/)
â¢ **Database**: [MongoDB](https://www.mongodb.com/) with [Mongoose ODM](https://mongoosejs.com/)
â¢ **Validation**: [Zod](https://zod.dev/) for request schema validation
â¢ **Security**: JWT for authentication, Bcrypt for password hashing

3. System Architecture

The project follows a Service-Oriented Architecture (SOA) with a focus on clean separation of concerns:

â¢ **Routes**: Define endpoints and apply middleware.
â¢ **Controllers**: Thin layer handling request/response logic.
â¢ **Services**: Contain the core business logic and interact with models.
â¢ **Models**: Mongoose schemas defining the data structure.
â¢ **Utils**: Reusable utility functions and custom error classes.

Interaction Flow

‘Client -> Route -> Middleware (Auth/Validation) -> Controller -> Service -> Model -> Database‘

4. Core Data Models

User (‘User‘)

â¢ ‘email‘ / ‘phone‘: Unique identifiers (one is required).
â¢ ‘password‘: Hashed credentials.
â¢ ‘role‘: ["user", "admin"].

Food (‘Food‘)

â¢ ‘name‘, ‘description‘, ‘price‘.
â¢ ‘category‘, ‘image‘.
â¢ ‘isAvailable‘: Boolean flag.

Cart (‘Cart‘)

â¢ ‘user‘: Reference to the User.
â¢ ‘items‘: Array of objects containing ‘food‘ reference and ‘quantity‘.

Order (‘Order‘)

â¢ ‘user‘, ‘items‘ (snapshot of name/price at order time).
â¢ ‘totalAmount‘, ‘status‘ (Pending, Preparing, etc.), ‘deliveryAddress‘.

5. API Endpoints Overview

Authentication

â¢ ‘POST /api/v1/auth/register‘: Register a new user.
â¢ ‘POST /api/v1/auth/login‘: Authenticate and receive a token.

Food

â¢ ‘GET /api/v1/food‘: List all available food items.
â¢ ‘POST /api/v1/food‘: Add new food (Admin only).

Cart

â¢ ‘GET /api/v1/cart‘: View current user’s cart.
â¢ ‘PATCH /api/v1/cart‘: Add/Update items in the cart.

Orders

â¢ ‘POST /api/v1/orders‘: Place an order from the cart.
â¢ ‘GET /api/v1/orders‘: Get user order history.

6. Middleware & Security

â¢ **Authentication**: ‘auth.middleware.js‘ verifies JWT and attaches user to the request.
â¢ **Authorization**: ‘authorize.middleware.js‘ restricts access based on user roles (e.g., admin).
â¢ **Validation**: ‘validate.js‘ uses Zod schemas to ensure incoming data is correct before reaching controllers.
â¢ **Logging**: ‘logger.middleware.js‘ logs incoming requests for monitoring.

7. Error Handling

The application uses a centralized error-handling strategy:

â¢ ‘AppError‘: A custom class inheriting from ‘Error‘ to handle operational errors with status codes.
â¢ ‘asyncHandler‘: (Used in routes) Wraps asynchronous functions to catch errors

and pass them to the global handler.
â¢ ‘error.middleware.js‘: The final middleware that sends a formatted JSON response to the client.

8. Environment Setup & Running

Prerequisites

â¢ [Node.js](https://nodejs.org/) (v16+ recommended)
â¢ [MongoDB](https://www.mongodb.com/) account or local instance

Installation

1. Clone the repository.
2. Install dependencies:
   ‘‘‘bash
   npm install
   ‘‘‘

Configuration

Create a ‘.env‘ file in the root directory and add the following variables:

```
PORT=5000
MONGODB_URI=your_mongodb_connection_string
JWT_SECRET=your_long_random_string
JWT_EXPIRES_IN=7d
```

Running the Project

â¢ **Production Mode**:
  ‘‘‘bash
  npm start
  ‘‘‘
â¢ **Development Mode** (with nodemon):
  ‘‘‘bash
  npm run dev
  ‘‘‘

9. Development Workflow

â¢ **Validation**: All new endpoints should have corresponding Zod validators in ‘src/validators‘.
â¢ **Logic**: Business logic must remain in ‘src/services‘, not in controllers.
â¢ **Testing**: (Planned) Unit and integration tests to ensure system stability.